2026-05-05-p1-f10-skill-system

P1-F10 — Skill System Implementation Plan

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Add agent-invoked Skills (`.helix/skills/*.md` with frontmatter `description/triggers/variables/requires_isolation`). Auto-invocation: agent loop checks user input against compiled trigger regexes, first match wins; rendered body injected as system message. Skills with `requires_isolation: true` run in F04 worktree. Surfaces: `/skills` slash + `helixcode skills` cobra.

Architecture: New `internal/commands/markdown_skills.go` (same package as F09 — reuses substRegex/tokenResolver). New `Skill` type, `SkillRegistry` (with `FindMatching`), `SkillLoader` (mirrors F09 MarkdownLoader). New `internal/agent/skill_dispatcher.go` with `Match(input) → (skill, captures, ok)`. Hot-reload via fsnotify (extend or sibling F09 watcher). Isolation routes through F04's `worktree.WorktreeManager`.

Tech Stack: Go 1.26, testify v1.11, gopkg.in/yaml.v3, fsnotify, cobra, zap. **No new external deps.**

Spec: `docs/superpowers/specs/2026-05-05-p1-f10-skill-system-design.md` (commit `5b80058`)

Working directory for `go` commands: `helix_code/`. Git from meta-repo root.

Anti-bluff smoke (FULL 4-term):

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" \
  internal/commands/markdown_skills.go internal/commands/
        skills_watcher.go \
  internal/commands/skills_command.go cmd/cli/skills_cmd.go \
  internal/agent/skill_dispatcher.go && echo BLUFF || echo clean
```

Task list

- [ ] P1-F10-T01 — bootstrap evidence + advance PROGRESS
- [ ] P1-F10-T02 — markdown_skills.go: Skill + SkillRegistry + parser + Render (TDD)
- [ ] P1-F10-T03 — SkillLoader: scan dirs + register/unregister (TDD)
- [ ] P1-F10-T04 — skills_watcher.go: fsnotify + debounce (TDD)

- [ ] P1-F10-T05 — agent/skill_dispatcher.go: Match + injection + isolation routing (TDD)
- [ ] P1-F10-T06 — /skills slash + helixcode skills cobra (TDD)
- [ ] P1-F10-T07 — main.go startup wiring + integration test (real fs, real triggers)
- [ ] P1-F10-T08 — Challenge with runtime evidence + cross-compile check
- [ ] P1-F10-T09 — Feature 10 close-out + push to 4 remotes

Task 1: Bootstrap evidence + advance PROGRESS

Append F10 evidence section header (spec `5b80058`, plan `<this commit>`, started 2026-05-05, status active). Replace current focus to F10. Insert F10 task list block (9 items) after F09 block (all 8 ticked). Commit `docs(P1-F10-T01): bootstrap Phase 1 / Feature 10 evidence + advance PROGRESS`.

Task 2: markdown_skills.go — Skill + SkillRegistry + parser + Render (TDD)

Files: create `helix_code/internal/commands/markdown_skills.go`, `helix_code/internal/commands/markdown_skills_test.go`.

Test cases (write FAILING first):

```
func TestParseSkillFile_Valid(t *testing.T) {
    body := `---
description: Refactor a React component
triggers:
  - "(?i)^refactor (.+) component$"
  - "(?i)^extract hook from (.+)"
variables:
  default_style: functional
requires_isolation: false
---

You are refactoring {{ARG1}}.`
    skill, err := parseSkillFile("refactor-button", body, "/tmp/
        skill.md")
    require.NoError(t, err)
    assert.Equal(t, "refactor-button", skill.Name())
    assert.Equal(t, "Refactor a React component",
        skill.Description())
    assert.Len(t, skill.triggers, 2)
    assert.False(t, skill.RequiresIsolation())
```

```
    assert.Contains(t, skill.body, "You are refactoring")
}

func TestParseSkillFile_BadRegexSkipped(t *testing.T) {
    body := `---
description: bad regex skill
triggers:
  - "[unclosed"
---

body`
    skill, err := parseSkillFile("bad", body, "/tmp/bad.md")
    require.NoError(t, err)
    // Bad regex is logged + skipped; skill still loads with
        valid triggers (here zero).
    assert.Empty(t, skill.triggers)
}

func TestSkill_Render_PositionalFromCapture(t *testing.T) {
    body := `---
description: x
triggers:
  - "^refactor (.+) component$"
---

Got: {{ARG1}}`
    skill, err := parseSkillFile("x", body, "/tmp/x.md")
    require.NoError(t, err)
    out, err := skill.Render([]string{"LoginButton"}, "", "")
    require.NoError(t, err)
    assert.Equal(t, "Got: LoginButton", out)
}

func TestSkill_Render_NamedCapture(t *testing.T) {
    body := `---
description: x
triggers:
  - "(?P<component>[A-Z][A-Za-z0-9]+) refactor"
variables:
  default_style: functional
---

Component: {{ARG.component}}, Style: {{ARG.default_style}}`
    skill, err := parseSkillFile("x", body, "/tmp/x.md")
    require.NoError(t, err)
    // captures supplied directly (the dispatcher does the regex
        extraction)
    captures := map[string]string{"component": "MyButton"}
```

```
    out, err := skill.RenderWithCaptures(nil, captures, "", "")
    require.NoError(t, err)
    assert.Contains(t, out, "Component: MyButton")
    assert.Contains(t, out, "Style: functional")
}

func TestSkillRegistry_FindMatching_FirstWins(t *testing.T) {
    reg := NewSkillRegistry()
    s1, _ := parseSkillFile("a", "---\ntriggers:\n  -
        \"^foo\"\n---\nA", "")
    s2, _ := parseSkillFile("b", "---\ntriggers:\n  -
        \"^foo\"\n---\nB", "")
    reg.Add(s1)
    reg.Add(s2)
    matched, _, ok := reg.FindMatching("foobar")
    require.True(t, ok)
    // First-registered (sorted by name) wins → "a"
    assert.Equal(t, "a", matched.Name())
}

func TestSkillRegistry_FindMatching_NamedCaptures(t *testing.T) {
    reg := NewSkillRegistry()
    s, _ := parseSkillFile("rc",
        "---\ntriggers:\n  - \"refactor (?P<comp>[A-Z][A-Za-z]+)
        component\"\n---\nbody {{ARG.comp}}", "")
    reg.Add(s)
    matched, captures, ok := reg.FindMatching("please refactor
        LoginButton component now")
    require.True(t, ok)
    assert.Equal(t, "rc", matched.Name())
    assert.Equal(t, "LoginButton", captures["comp"])
}

func TestSkillRegistry_AddRemove(t *testing.T) {
    reg := NewSkillRegistry()
    s, _ := parseSkillFile("x", "---\ntriggers:\n  - \"^x\"\n---
        \nbody", "")
    reg.Add(s)
    _, ok := reg.Get("x")
    require.True(t, ok)
    reg.Remove("x")
    _, ok = reg.Get("x")
    assert.False(t, ok)
}
```

Implementation (uses F09's `substRegex`, `tokenResolver` indirectly via a Skill-specific render):

```
package commands

import (
    "fmt"
    "regexp"
    "sort"
    "strings"
    "sync"

    "gopkg.in/yaml.v3"
)

type Skill struct {
    name              string
    description       string
    body              string
    variables         map[string]string
    triggerPatterns   []string
    triggers          []*regexp.Regexp
    requiresIsolation bool
    sourcePath        string
}

type skillFrontmatter struct {
    Description       string            `yaml:"description"`
    Triggers          []string          `yaml:"triggers"`
    Variables         map[string]string `yaml:"variables"`
    RequiresIsolation bool
        `yaml:"requires_isolation"`
}

func (s *Skill) Name() string                  { return s.name }
func (s *Skill) Description() string           { return
        s.description }
func (s *Skill) SourcePath() string            { return
        s.sourcePath }
func (s *Skill) RequiresIsolation() bool       { return
        s.requiresIsolation }
func (s *Skill) Body() string                  { return s.body }

// parseSkillFile parses a Markdown file (with required
        frontmatter) into a Skill.
// Bad regex patterns are logged and skipped; the skill loads
        with the remaining
// valid triggers.
func parseSkillFile(name, raw, sourcePath string) (*Skill,
        error) {
```

```
    s := &Skill{name: name, sourcePath: sourcePath, variables:
        map[string]string{}}
    body := raw
    if strings.HasPrefix(raw, "---\n") {
        end := strings.Index(raw[4:], "\n---")
        if end == -1 {
            return nil, fmt.Errorf("skill %s: unterminated
        frontmatter", name)
        }
        fm := raw[4 : 4+end]
        body = strings.TrimPrefix(raw[4+end+4:], "\n")
        var meta skillFrontmatter
        if err := yaml.Unmarshal([]byte(fm), &meta); err != nil {
            return nil, fmt.Errorf("skill %s: parse frontmatter:
        %w", name, err)
        }
        s.description = meta.Description
        s.triggerPatterns = meta.Triggers
        s.requiresIsolation = meta.RequiresIsolation
        if meta.Variables != nil {
            s.variables = meta.Variables
        }
        for _, p := range meta.Triggers {
            re, err := regexp.Compile(p)
            if err != nil {
                continue // bad regex skipped (logged at loader
        level)
            }
            s.triggers = append(s.triggers, re)
        }
    }
    s.body = strings.TrimSpace(body)
    return s, nil
}

// Render fills the body using positional args + Skill.variables.
func (s *Skill) Render(args []string, selection, currentFile
        string) (string, error) {
    cc := &CommandContext{Args: args, Selection: selection,
        CurrentFile: currentFile}
    return (&MarkdownCommand{name: s.name, body: s.body,
        variables: s.variables}).render(cc)
}

// RenderWithCaptures renders with named captures (override
        variables map).
```

```
func (s *Skill) RenderWithCaptures(args []string, captures
        map[string]string, selection, currentFile string)
        (string, error) {
    merged := make(map[string]string, len(s.variables)
        +len(captures))
    for k, v := range s.variables {
        merged[k] = v
    }
    for k, v := range captures {
        merged[k] = v
    }
    cc := &CommandContext{Args: args, Selection: selection,
        CurrentFile: currentFile}
    return (&MarkdownCommand{name: s.name, body: s.body,
        variables: merged}).render(cc)
}

// SkillRegistry stores skills and matches user input to
        triggers.
type SkillRegistry struct {
    mu     sync.RWMutex
    skills map[string]*Skill
}

func NewSkillRegistry() *SkillRegistry {
    return &SkillRegistry{skills: map[string]*Skill{}}
}

func (r *SkillRegistry) Add(s *Skill) {
    r.mu.Lock()
    defer r.mu.Unlock()
    r.skills[s.Name()] = s
}

func (r *SkillRegistry) Remove(name string) {
    r.mu.Lock()
    defer r.mu.Unlock()
    delete(r.skills, name)
}

func (r *SkillRegistry) Get(name string) (*Skill, bool) {
    r.mu.RLock()
    defer r.mu.RUnlock()
    s, ok := r.skills[name]
    return s, ok
}

func (r *SkillRegistry) List() []*Skill {
```

```
    r.mu.RLock()
    defer r.mu.RUnlock()
    names := make([]string, 0, len(r.skills))
    for n := range r.skills {
        names = append(names, n)
    }
    sort.Strings(names)
    out := make([]*Skill, 0, len(names))
    for _, n := range names {
        out = append(out, r.skills[n])
    }
    return out
}

// FindMatching returns the first skill (lex-sorted by name)
        whose trigger
// regex matches input, plus the named-capture groups extracted
        from the
// match. Positional capture groups (numbered 1..N) become args
        at indexes 1-N.
func (r *SkillRegistry) FindMatching(input string) (*Skill,
        map[string]string, bool) {
    r.mu.RLock()
    defer r.mu.RUnlock()
    names := make([]string, 0, len(r.skills))
    for n := range r.skills {
        names = append(names, n)
    }
    sort.Strings(names)
    for _, n := range names {
        s := r.skills[n]
        for _, re := range s.triggers {
            m := re.FindStringSubmatch(input)
            if m == nil {
                continue
            }
            caps := map[string]string{}
            for i, name := range re.SubexpNames() {
                if name == "" || i >= len(m) {
                    continue
                }
                caps[name] = m[i]
            }
            return s, caps, true
        }
    }
    return nil, nil, false
}
```

Run TDD cycle. Subject: `feat(P1-F10-T02): Skill + SkillRegistry + parser + Render`.

Task 3: SkillLoader — scan dirs + register/unregister (TDD)

Mirror F09's `MarkdownLoader` pattern. Tests: `LoadProjectAndUser` (project overrides user), `ReloadDiff` (added/removed/updated), `BadFrontmatterSkipped`, `NonExistentDirsSkipped`, `Loaded` snapshot. Implementation appended to `markdown_skills.go`. Subject: `feat(P1-F10-T03): SkillLoader scans .helix/skills dirs and registers/unregisters`.

Task 4: skills_watcher.go — fsnotify + debounce (TDD)

Mirror F09's `MarkdownWatcher` pattern. Implementation in `internal/commands/skills_watcher.go`. Tests: `DebouncesAndReloads`, `StopsOnContextCancel`, `HandlesMissingDirGracefully`. Subject: `feat(P1-F10-T04): skills_watcher.go fsnotify + debounced reload`.

Task 5: agent/skill_dispatcher.go — Match + injection + isolation routing (TDD)

Files: create `helix_code/internal/agent/skill_dispatcher.go`, `helix_code/internal/agent/skill_dispatcher_test.go`.

```
package agent

import (
    "context"
    "fmt"

    "dev.helix.code/internal/commands"
    "dev.helix.code/internal/tools/worktree"
)

// SkillDispatcher routes user input to skills via trigger
        matching.
type SkillDispatcher struct {
    registry *commands.SkillRegistry
    wtMgr    *worktree.Manager // optional; nil disables
        isolation
}

func NewSkillDispatcher(reg *commands.SkillRegistry, wtMgr
        *worktree.Manager) *SkillDispatcher {
    return &SkillDispatcher{registry: reg, wtMgr: wtMgr}
}
```

```
// Match looks up a skill matching the user input. Returns the
        rendered system
// message to inject (empty if no match), whether isolation is
        required, and
// the matched skill (nil on no match).
func (d *SkillDispatcher) Match(ctx context.Context, input,
        selection, currentFile string) (rendered string, matched
        *commands.Skill, captures map[string]string, ok bool,
        err error) {
    if d.registry == nil {
        return "", nil, nil, false, nil
    }
    skill, caps, found := d.registry.FindMatching(input)
    if !found {
        return "", nil, nil, false, nil
    }
    out, rerr := skill.RenderWithCaptures(nil, caps, selection,
        currentFile)
    if rerr != nil {
        return "", skill, caps, false, fmt.Errorf("skill %s
        render: %w", skill.Name(), rerr)
    }
    return out, skill, caps, true, nil
}
```

Tests: - `TestSkillDispatcher_Match_Injects` — registry with a real skill, Match returns the rendered body with captures substituted. - `TestSkillDispatcher_NoMatch` — registry but input doesn't match → `ok=false`, empty rendered. - `TestSkillDispatcher_RegistryNil` — `ok=false`, no error. - `TestSkillDispatcher_RequiresIsolation_FlaggedInResult` — assert returned skill exposes `RequiresIsolation()` so caller can route to worktree.

Subject: `feat(P1-F10-T05): agent/skill_dispatcher.go Match + capture extraction`.

Task 6: /skills slash + helixcode skills cobra (TDD)

Files: `internal/commands/skills_command.go`, `cmd/cli/skills_cmd.go`, plus `_test.go` siblings + `internal/commands/builtin/skills_register_test.go`.

`/skills`: list / show / invoke / reload / unknown. `helixcode skills`: list / show / invoke / reload.

`invoke` runs the skill explicitly (bypassing trigger matching) — `cmd.Execute(ctx, &CommandContext{Args: args})`. Mirror F09's pattern exactly.

Subject: `feat(P1-F10-T06): /skills slash + helixcode skills cobra`.

Task 7: cmd/cli/main.go startup wiring + integration test

Add to main.go after F09 wiring:

```
// F10: Skills
skillReg := commands.NewSkillRegistry()
skillProjectDir := filepath.Join(".", ".helix", "skills")
var skillUserDir string
if userCfg, err := os.UserConfigDir(); err == nil {
    skillUserDir = filepath.Join(userCfg, "helixcode", "skills")
}
skillLoader := commands.NewSkillLoader(skillReg,
        skillProjectDir, skillUserDir)
if logger != nil { skillLoader.SetLogger(logger) }
if err := skillLoader.Load(); err != nil { log.Printf("skills:
        load failed: %v", err) }
if w, err := commands.NewSkillsWatcher(skillLoader,
        []string{skillProjectDir, skillUserDir}); err == nil {
    if logger != nil { w.SetLogger(logger) }
    go w.Run(ctx)
    defer w.Close()
}
skillDispatcher := agent.NewSkillDispatcher(skillReg,
        worktreeMgr) // worktreeMgr from F04
// pass skillDispatcher to baseAgent (existing wiring point)
if err :=
        cmdRegistry.Register(commands.NewSkillsCommand(skillLoader,
        skillReg)); err != nil {
    log.Printf("skills: register slash failed: %v", err)
}
```

Cobra dispatcher block (mirrors F09's pattern, keyed on `os.Args[1] == "skills"`).

Integration test `tests/integration/skills_test.go` (`//go:build integration`): - `TestSkills_LoadAndAutoMatch` — write `.helix/skills/refactor.md` with trigger `^refactor (.+) component$`; FindMatching("refactor LoginButton component") returns skill + capture `LoginButton`; Render produces body containing `LoginButton`. - `TestSkills_RequiresIsolation_FlagPresent` — confirm `RequiresIsolation()=true` skill is exposed correctly to caller.

Subject: `feat(P1-F10-T07): wire skills loader + watcher + dispatcher into main.go + integration test.`

Task 8: Challenge with runtime evidence + cross-compile check

Harness `tests/integration/cmd/p1f10_challenge/main.go`: 1. Create tempdir `.helix/skills/refactor.md` with frontmatter `triggers: ["^refactor (?`

P<comp>[A-Z][A-Za-z]+) component$"], body Refactoring {{ARG.comp}} 2. Construct registry + loader + dispatcher 3. Call dispatcher.Match("refactor LoginButton component") → assert match, capture group comp=LoginButton, rendered body Refactoring LoginButton 4. Mutate file: change body to Now: {{ARG.comp}}, reload, re-match, assert new body 5. Anti-bluff smoke clean 6. Cross-compile linux clean

run.sh + CHALLENGE.md in challenges/p1-f10-skills/. Dual commit (submodule + meta-repo). Append verbatim run log to 06_phase_1_evidence.md.

Subject: feat(P1-F10-T08): challenge with runtime evidence + cross-compile check.

Task 9: Feature 10 close-out + push to 4 remotes

Tick 9 task items. Advance PROGRESS to idle (F11 candidate). Final unit + integration + smoke + cross-compile + go vet + challenge re-run. Commit chore(P1-F10-T09): Feature 10 (Skill System) close-out. Push origin/github/gitlab/upstream non-force.

Self-review notes

1. Spec coverage: all 10 spec sections mapped to tasks (T02 types/render, T03 loader, T04 watcher, T05 dispatcher/isolation, T06 surfaces, T07 wiring, T08 challenge, T09 close-out).
2. Type consistency: Skill, SkillRegistry, SkillLoader, SkillsWatcher, SkillDispatcher, parseSkillFile, FindMatching, RequiresIsolation consistent across tasks.
3. Cross-platform: pure Go; fsnotify cross-platform.
4. Anti-bluff: full 4-term smoke + Challenge captures real Match → Render output.
5. No new external deps.
6. Branch + push: stays on main, non-force to all four remotes.